

C++ Multiple Inheritance

Grounded in ECM2433 inheritance material. Focuses on inherited access, multiple inheritance syntax, the diamond problem, virtual base classes, virtual member functions, and runtime dispatch through base-class pointers.

What You Need To Know

1. Inherited access depends on the inheritance specifier

The access specifier in the class head changes how inherited public and protected members appear inside the derived class. Private members of the base are still private to the base and cannot be accessed directly by derived code.

- **Public inheritance:** inherited public stays public; inherited protected stays protected.
- **Protected inheritance:** inherited public and protected become protected.
- **Private inheritance:** inherited public and protected become private.
- A class inherits privately by default. A struct inherits publicly by default.

```
class Base {
    int private_value;
protected:
    int protected_value;
public:
    void set(int a, int b) { private_value = a; protected_value = b; }
};

class Derived : protected Base {
public:
    void reset() {
        protected_value = 0; /* OK: protected member */
        set(1, 2);           /* OK inside Derived */
        /* private_value = 3; ERROR: private to Base */
    }
};

int main() {
    Derived d;
    d.reset(); /* OK */
    d.set(1, 2); /* ERROR: set became protected in Derived */
}
```

2. Multiple inheritance lists more than one base

A derived class can inherit from several base classes by separating them with commas. The derived object then contains base-class subobjects for each base.

```
class HasX {
protected:
    int x;
public:
    void show_x() const { std::cout << x << "\n"; }
};

class HasY {
protected:
    int y;
public:
    void show_y() const { std::cout << y << "\n"; }
};

class Point : public HasX, public HasY {
public:
    void set(int new_x, int new_y) { x = new_x; y = new_y; }
};
```

3. The diamond problem creates duplicate base subobjects

If two intermediate bases both inherit from the same base, the final derived class can receive two separate copies of that shared base. Unqualified access to a member of the shared base then becomes ambiguous.

```
class Base { public: int value; };
class Left : public Base {};
class Right : public Base {};
class Bottom : public Left, public Right {};

int main() {
    Bottom b;
```

```

        b.value = 10;           /* ERROR: Left::Base::value or Right::Base::value? */
        b.Left::value = 10;    /* OK: explicitly selects one copy */
    }

```

4. Virtual base classes share one base subobject

Use virtual inheritance on the intermediate classes when the final class should contain only one shared copy of the common base. The most-derived class is then responsible for constructing that virtual base.

```

class Base { public: int value; };
class Left : virtual public Base {};
class Right : virtual public Base {};
class Bottom : public Left, public Right {};

int main() {
    Bottom b;
    b.value = 10;           /* OK: only one Base subobject */
}

```

5. Virtual functions choose the override at runtime

A virtual member function allows derived classes to override behaviour. When the function is called through a base pointer or reference, C++ uses the actual object type at runtime to choose the correct override.

```

class Animal {
protected:
    std::string name;
public:
    Animal(std::string n) : name(n) {}
    virtual void speak() const { std::cout << name << "\n"; }
};

class Dog : public Animal {
public:
    Dog(std::string n) : Animal(n) {}
    void speak() const override { std::cout << "Bark " << name << "\n"; }
};

Dog pet("Fido");
Animal *a = &pet;
a->speak();           /* Bark Fido */

```

6. Constructor order and virtual bases

Base constructors run before the derived constructor body. With virtual bases, the most-derived class constructs the virtual base directly, even if intermediate classes also mention it in their initializer lists. Destructors run in the reverse order.

```

class Animal {
public:
    Animal(std::string name) {}
    virtual void speak() const = 0;
    virtual ~Animal() = default;
};

class Wolf : virtual public Animal {
public:
    Wolf(std::string name) : Animal(name) {}
};

class Poodle : virtual public Animal {
public:
    Poodle(std::string name) : Animal(name) {}
};

class Dog : public Wolf, public Poodle {
public:
    Dog(std::string name) : Animal(name), Wolf(name), Poodle(name) {}
    void speak() const override {}
};

```

Practice Questions

Questions 1 to 3 use this code:

```

class Base {
    int i;
protected:
    int j;
public:
    void set(int a, int b) { i = a; j = b; }
    void show() const { std::cout << i << " " << j << "\n"; }
}

```

```

};

class Derived : public Base {
public:
    void reset() { j = 0; }
    void show_all() const { show(); }
};

int main() {
    Derived obj;
    obj.set(1, 2);      /* line A */
    obj.show();         /* line B */
    obj.show_all();     /* line C */
    obj.i = 5;         /* line D */
}

```

- 1 For lines A to D, identify which compile and which do not.
- 2 Change the inheritance line to `class Derived : protected Base`. Now identify which of lines A to D compile.
- 3 In `reset`, explain why `j = 0` is allowed but `i = 0` would not compile.

Questions 4 to 6 use this code:

```

class Base { public: int value; };
class Left : public Base {};
class Right : public Base {};
class Bottom : public Left, public Right {};

int main() {
    Bottom b;
    b.value = 10;      /* line E */
}

```

- 4 Explain why line E fails to compile.
- 5 Rewrite line E so it selects the value inherited through `Left`.
- 6 Change the class definitions so `b.value = 10;` compiles without needing scope resolution.

Questions 7 to 10 use this code:

```

class Animal {
protected:
    std::string name;
public:
    Animal(std::string n) : name(n) {}
    virtual void speak() const { std::cout << name << "\n"; }
};

class Dog : public Animal {
public:
    Dog(std::string n) : Animal(n) {}
    void speak() const override { std::cout << "Bark " << name << "\n"; }
};

int main() {
    Dog pet("Fido");
    pet.speak();        /* line F */
    Animal *a = &pet;
    a->speak();          /* line G */
}

```

- 7 Trace lines F and G. What does each line print?
- 8 Remove `virtual` from `Animal::speak`. What does line G print now?
- 9 Make `Animal::speak` pure virtual. What happens if someone writes `Animal a("Rex");`?
- 10 Add the destructor declaration `Animal` should usually have when it is used through base-class pointers.

Mark Scheme

Attempt the questions before checking. Equivalent wording is fine if it correctly uses inherited access, virtual bases, and virtual dispatch.

- 1 A, B, and C compile. D does not compile because i is private in Base.
- 2 With protected inheritance, A and B no longer compile from main because set and show become protected in Derived. C still compiles because show_all is public in Derived. D still does not compile.
- 3 j is protected, so Derived member functions can access it. i is private, so only Base member functions can access it directly.
- 4 Bottom contains two Base subobjects: one through Left and one through Right. The expression b.value is ambiguous.
- 5 Use `b.Left::value = 10;`.
- 6 Make the intermediate inheritance virtual: `class Left : virtual public Base {};` and `class Right : virtual public Base {};`.
- 7 Line F prints Bark Fido. Line G also prints Bark Fido because speak is virtual.
- 8 Without virtual, line G uses the base-pointer type and prints Fido.
- 9 Animal becomes abstract, so `Animal a("Rex");` does not compile.
- 10 Use `virtual ~Animal() = default;`.

Useful Code Patterns

```
/* Multiple inheritance */
class Derived : public Base1, public Base2 {};

/* Diamond fix */
class Left : virtual public Base {};
class Right : virtual public Base {};
class Bottom : public Left, public Right {};

/* Virtual dispatch */
class Base {
public:
    virtual void f() const = 0;
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void f() const override {}
};

Base *ptr = &derived_object;
ptr->f();      /* calls Derived::f at runtime */
```